

```

double x = 1.003;
// Different n sizes and test it
int n_array[] = {2500, 5000, 10000, 20000, 40000, 80000};
size_t lengthOfSize = sizeof(n_array) / sizeof(n_array[0]);

for (int s = 0; s < lengthOfSize; s++) {
    int n = n_array[s];
    int trials = 50000;

    printf("==== n = %d ==== \n", n);

    // Method 1
    clock_t start = clock();
    for (int i = 0; i < trials; i++) {
        Method1(x, n);
    }
    clock_t end = clock();
    double avg_time = ((double)(end - start) / CLOCKS_PER_SEC)
/ trials;

    printf("Method 1; n = %d, time = %.12f seconds \n", n,
avg_time);

    // Method 2
    start = clock();
    for (int i = 0; i < trials; i++) {
        Method2(x, n);
    }
    end = clock();
    avg_time = ((double)(end - start) / CLOCKS_PER_SEC) / trials;
    printf("Method 2; n = %d, time = %.12f seconds \n", n, avg_time);

    // Method 3
    start = clock();
    for (int i = 0; i < trials; i++) {
        Method3(x, n);
    }
    end = clock();
    avg_time = ((double)(end - start) / CLOCKS_PER_SEC) / trials;
    printf("Method 3; n = %d, time = %.12f seconds \n", n, avg_time);
}

```

Analyse av Methodene

Metode 1

```
double Method1(double x, int n){
    if (n == 1){
        return x;
    } else {
        return x * Method1(x, n-1);
    }
}
```

Dette er en rekursiv funksjon gjør en multiplikasjon og en rekursiv kall med eksponente -1 med seg (n-1). Hvis eksponente er 1 slutt med kallingen:

$$T(1) = c$$

$$T(n) = T(n - 1) + c$$

Her er c de konstante operasjoner som kan kjøres hver gang (multiplikasjon eller/og sammenligningen). Utvider vi det får vi at

$$T(n) = T(1) + (n - 1)c$$

Det gjør minst en multiplikasjon eller/og sammenligning operasjon og en til for eksponentent >2. Dette betyr at algoritmen er linært:

$$T(n) = O(n)$$

Metode 2

```
if (n == 0){
    return 1;
}
if (n == 1){
    return x;
}

double new_x = x*x;
if (n & 1){
    int new_n = (n-1)/2;
    return x*Method2(new_x,new_n);
} else {
    int new_n = n/2;
    return Method2(new_x,new_n);
}
```

Denne formelen gjør alltid en konstant arbeid, nemlig regningen av ny x-verdi, og kaller seg selv med halvparten av n.

$$T(n) = T(n/2) + c$$

Når vi utvider den finner vi ut at n deles med to k antall ganger:

$$T(n) = T(n/2) + c = T(n/4) + 2c = T(n/8) + 3c + \dots$$

$$n/2^k$$

Rekursjonen stopper ved 1. Altså $n/2^k \approx 1$

Det betyr at tidskompleksitet vårt er:

$$T(n) = T(1) + k * c = O(\log(n))$$

Metode 3

```
double Method3(double x, int n){  
    return pow(x,n);  
}
```

Dette metoden er en innebygd funksjon og bruker bare et call uansett av datamengde. Tidskomplekstitet blir bare

$$O(1)$$

Testing

Vanlig tidstesting gjennom å starte klokka, kjøre funksjonen og til slutt stoppe klokka, går for altfor fort. Derfor kjører vi flere iterasjoner av funksjonen deler totale tiden med antall iterasjoner for å få gjennomsnittlig tid. See koden under for å få bedre forståelse:

```
double x = 1.003;  
// Different n sizes and test it  
int n_array[] = {2500, 5000, 10000, 20000, 40000, 80000};  
size_t lengthOfSize = sizeof(n_array) / sizeof(n_array[0]);  
  
for (int s = 0; s < lengthOfSize; s++) {  
    int n = n_array[s];  
    int trials = 50000;
```

```

printf("==== n = %d =====\n", n);

// Method 1
clock_t start = clock();
for (int i = 0; i < trials; i++) {
    Method1(x, n);
}
clock_t end = clock();
double avg_time = ((double)(end - start) / CLOCKS_PER_SEC) / trials;
printf("Method 1; n = %d, time = %.12f seconds\n", n, avg_time);

// Method 2 og osv...
}

```

```

==== n = 2500 ====
Method 1; n = 2500, time = 0.000032200000 seconds
Method 2; n = 2500, time = 0.000000040000 seconds
Method 3; n = 2500, time = 0.000000020000 seconds
==== n = 5000 ====
Method 1; n = 5000, time = 0.000065020000 seconds
Method 2; n = 5000, time = 0.000000040000 seconds
Method 3; n = 5000, time = 0.000000040000 seconds
==== n = 10000 ====
Method 1; n = 10000, time = 0.000133860000 seconds
Method 2; n = 10000, time = 0.000000040000 seconds
Method 3; n = 10000, time = 0.000000000000 seconds
==== n = 20000 ====
Method 1; n = 20000, time = 0.000269520000 seconds
Method 2; n = 20000, time = 0.000000060000 seconds
Method 3; n = 20000, time = 0.000000020000 seconds
==== n = 40000 ====
Method 1; n = 40000, time = 0.000546900000 seconds
Method 2; n = 40000, time = 0.000000060000 seconds
Method 3; n = 40000, time = 0.000000020000 seconds

```

Tolkning

Ved metode 1 ser vi klar fordobling av tid hver gang datamengden doubles. Dette stemmer ved analysen av tidskompleksitet, nemlig $O(n)$

Ved metode 3 ser vi også en konstant tidsbruk uavhengig av datamengden. Dette stemmer siden metode bare gjør et direkte kall til biblioteket og bruker ikke rekursjon.

Ved metode 2 ser vi noe interessant. Vi ser en liten økning mellom 10000 og 20000. Dette kan være på grunn av hvordan håndterer rekursjon. Siden metoden deler eksponent i to og stopper rekursjonen ved 0 og 1, handler det om hvor mange delinger

av eksponent kommer frem til de verdiene. Ved 10000 tar det 14 ganger siden $2^{14} = 16384$. Ved 20000 tar det 15 ganger siden $2^{15} = 32768$. Men tar en til rekursiv kall 0.2 microsekunder? Ja. Det vi kan konkludere med er at ved større og større datamengde kan håndteres bare med en rekursiv kall. Dette stemmer med analysen av tidskomplekistet som reflekterer denne effekten.